

Problems in Java Multithreading

Multithreading allows multiple threads to make progress within the same program. This improves responsiveness and performance, but it also creates problems when threads access the same mutable data without proper coordination.

The major multithreading problems are:

1. Race conditions
2. Atomicity problems
3. Visibility problems
4. Ordering problems
5. Thread interference
6. Data inconsistency

These concepts are closely connected, but they are not exactly the same.

1. Shared Resources

A **shared resource** is any data or object that can be accessed by more than one thread.

Examples:

```
int count = 0;
List<Integer> numbers;
BankAccount account;
```

Common shared resources include:

- Objects stored in heap memory
- Static variables
- Shared collections
- Files

- Database connections
- Network resources

A resource becomes dangerous when it is both:

```
Shared
+
Mutable
```

Immutable data can be shared safely because its state cannot change after creation.

2. Critical Section

A **critical section** is the part of the program that reads or modifies shared mutable data and therefore requires controlled access.

```
class Counter {
    int count = 0;        // shared resource

    void increment() {
        count++;          // critical section
    }
}
```

Here:

```
Shared resource → count
Critical section → count++
```

A critical section is the location in code where incorrect thread interleaving can cause a problem.

The critical section may contain a single statement or several related statements.

Example:

```
if (balance >= amount) {  
    balance = balance - amount;  
}
```

Both the condition check and the update belong to the same logical critical section.

Race Condition

3. What Is a Race Condition?

A **race condition** occurs when the correctness of a program depends on the unpredictable timing or execution order of multiple threads.

In simple terms:

Multiple threads race to access or modify the same shared state, and the final result depends on which thread executes first.

Consider:

```
count++;
```

It looks like one Java statement, but conceptually it performs multiple operations:

1. Read count
2. Add 1
3. Write the updated value

Because these steps can be interleaved with another thread, `count++` is not atomic.

4. Simple Race Condition Example

```
class Counter {  
    int count = 0;
```

```
void increment() {
    count++;
}
```

Suppose two threads call `increment()` when:

```
count = 0
```

The expected execution is:

```
Thread-1: 0 → 1
Thread-2: 1 → 2
```

Expected final result:

```
count = 2;
```

However, the following interleaving is possible:

Step	Thread-1	Thread-2
1	Reads <code>count = 0</code>	
2		Reads <code>count = 0</code>
3	Calculates <code>1</code>	
4		Calculates <code>1</code>
5	Writes <code>1</code>	
6		Writes <code>1</code>

Final result:

```
count = 1;
```

Two increments were attempted, but one update was lost.

This is a race condition.

5. Complete Race Condition Program

```
class Counter {  
    int count = 0;  
  
    void increment() {  
        count++;  
    }  
}
```

```
public class RaceConditionDemo {  
  
    public static void main(String[] args)  
        throws InterruptedException {  
  
        Counter counter = new Counter();  
  
        Thread t1 = new Thread(() -> {  
            for (int i = 1; i <= 100_000; i++) {  
                counter.increment();  
            }  
        });  
  
        Thread t2 = new Thread(() -> {  
            for (int i = 1; i <= 100_000; i++) {  
                counter.increment();  
            }  
        });  
  
        t1.start();  
        t2.start();  
    }  
}
```

```
t1.join();
t2.join();

System.out.println(counter.count);
}
}
```

Expected result:

```
200000
```

A possible result may be:

```
173482
```

The exact incorrect value is not guaranteed. It depends on thread scheduling and the number of overlapping updates.

A race condition is often non-deterministic:

```
The program may produce the correct result sometimes
and an incorrect result at other times.
```

This is what makes race conditions difficult to detect and reproduce.

Atomicity

6. What Is Atomicity?

An operation is **atomic** when it appears to other threads as one indivisible action. No other thread can observe or interfere with the operation halfway through.

```
Atomic operation:
Starts
```

→ completes as one unit
→ other threads do not see an intermediate state

Atomicity in multithreading is about preventing unsafe interleaving.

7. Why Atomicity Matters

A thread can be paused by the scheduler between individual operations.

If a task contains several dependent steps and those steps are not protected, another thread can interfere between them.

Consider:

```
count++;
```

Conceptually:

```
int temporary = count;  
temporary = temporary + 1;  
count = temporary;
```

Because these are separate read, modify, and write operations, another thread can run between them.

The race condition in the counter example is fundamentally caused by a lack of atomicity.

8. Check-Then-Act Atomicity Problem

Consider a bank withdrawal:

```
void withdraw(int amount) {  
    if (balance >= amount) {  
        balance = balance - amount;  
    }  
}
```

The method performs two related actions:

1. Check whether enough balance exists
2. Deduct the amount

Suppose:

```
balance = 1000
```

Two threads attempt to withdraw `800`.

Possible execution:

```
Thread-1 checks balance → sufficient  
Thread-2 checks balance → sufficient  
Thread-1 deducts 800  
Thread-2 deducts 800
```

Both threads made their decision using the old balance.

The condition check and update must therefore be treated as one atomic operation.

9. Real-World Atomicity Example

Consider a money transfer:

1. Deduct money from Account A
2. Add money to Account B

If only the first step completes, the system becomes inconsistent.

In concurrent programming, another thread should not observe the system in a partially completed state where money has been deducted but not yet credited.

The complete transfer should be protected as one logical operation.

10. Solving Atomicity Problems

Common solutions include:

Using **synchronized**

```
synchronized void increment() {  
    count++;  
}
```

Using atomic classes

```
AtomicInteger count = new AtomicInteger();  
  
count.incrementAndGet();
```

Using explicit locks

```
Lock lock = new ReentrantLock();
```

The correct choice depends on the complexity of the operation.

Atomic and Non-Atomic Operations

11. Simple Reads and Writes

A simple read or write of most Java primitive values and object references is atomic.

Examples:

```
int x = 10;  
x = 20;
```

```
int y = x;
```

```
Object value = new Object();
```

Another thread will not observe half of an `int` value or half of an object reference.

However, atomicity alone does not guarantee visibility.

A write may be atomic but still not become visible to another thread without a proper happens-before relationship.

12. Operations That Are Not Atomic

Increment and Decrement

```
count++;  
count--;
```

These require:

```
Read  
→ modify  
→ write
```

Compound Assignment

```
x += 5;
```

This is conceptually:

```
x = x + 5;
```

It contains a read and a write, so it is not atomic.

Check-Then-Act Logic

```
if (balance > 0) {  
    withdraw();  
}
```

The check and action are separate.

Another thread can change `balance` after the check but before the action.

Compound Collection Operations

A method call such as:

```
list.add(1);  
map.put("key", "value");
```

may perform several internal operations.

Whether the method is thread-safe depends on the collection implementation.

For example:

```
ArrayList      → not thread-safe  
HashMap        → not thread-safe  
ConcurrentHashMap → designed for concurrent access
```

Even with thread-safe collections, a sequence of multiple method calls may still require additional synchronization.

Example:

```
if (!map.containsKey(key)) {  
    map.put(key, value);  
}
```

The complete check-then-put operation is not automatically atomic unless an atomic API such as `putIfAbsent()` is used.

13. The `long` and `double` Case

The Java Language Specification permits a non-volatile `long` or `double` read or write to be treated as two separate 32-bit operations.

```
long value = 100L;  
double amount = 50.5;
```

On most modern JVMs and processors, these operations are implemented atomically. However, portable Java code should not depend on that implementation detail.

Declaring them `volatile` guarantees atomic reads and writes:

```
volatile long value;  
volatile double amount;
```

This still does not make compound operations atomic:

```
value++;
```

Even when `value` is volatile, `value++` remains a read-modify-write sequence.

Visibility

14. What Is a Visibility Problem?

A visibility problem occurs when:

One thread updates shared data, but another thread does not reliably observe the updated value.

The first thread may have performed the write correctly. The problem is that another thread has no synchronization guarantee requiring it to see that write.

15. Why Visibility Problems Occur

Modern systems improve performance through:

- CPU caches
- Processor registers
- Compiler optimizations
- JVM optimizations
- Instruction reordering

A simplified mental model is:

```
Thread
↓
CPU registers and caches
↓
Shared memory
```

This model is useful for understanding the problem, but the Java Memory Model does not define visibility as simply reading directly from RAM or CPU cache.

The important rule is:

Without a happens-before relationship, one thread is not guaranteed to observe another thread's write.

16. Visibility Problem Example

```
class VisibilityDemo {
    static boolean flag = false;

    public static void main(String[] args) {

        Thread updater = new Thread(() -> {
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });
```

```

        return;
    }

    flag = true;
    System.out.println("Flag updated to true");
});

Thread observer = new Thread(() -> {
    while (!flag) {
        // waiting
    }

    System.out.println("Detected flag change");
});

updater.start();
observer.start();
}
}

```

Expected behavior:

```

Updater changes flag to true
→ observer exits the loop

```

However, without proper synchronization, the observer is not guaranteed to detect the update.

A possible result is:

```

Flag updated to true

```

and the observer may continue looping.

The write happened, but another thread did not reliably observe it.

volatile

17. Using **volatile** for Visibility

The variable can be declared as volatile:

```
static volatile boolean flag = false;
```

A volatile write happens-before every subsequent volatile read of the same variable.

Therefore:

```
Thread-1 writes flag = true  
→ Thread-2 later reads flag  
→ Thread-2 must observe the updated value
```

volatile also prevents certain reorderings around volatile reads and writes.

18. What **volatile** Guarantees

volatile provides:

1. Visibility
2. Ordering guarantees around the volatile access
3. Atomic reads and writes of the volatile variable itself

It is best suited for simple state communication where one thread writes a value and other threads read it.

Examples:

```
volatile boolean running;  
volatile boolean shutdownRequested;  
volatile int currentState;
```

19. What **volatile** Does Not Guarantee

volatile does not make compound operations atomic.

```
volatile int count = 0;  
  
count++;
```

The increment still performs:

```
Read count  
→ add 1  
→ write count
```

Two threads can still read the same old value and overwrite one another.

Therefore:

```
volatile → visibility and ordering  
volatile ≠ compound-operation atomicity
```

For atomic increments, use:

```
AtomicInteger count = new AtomicInteger();  
  
count.incrementAndGet();
```

or protect the operation with synchronization.

Visibility Through Synchronization

20. How **synchronized** Provides Visibility

Monitor operations create happens-before relationships.

Thread-1 releases a monitor
→ Thread-2 later acquires the same monitor
→ Thread-2 can observe writes made before Thread-1 released it

Example:

```
synchronized (lock) {  
    count++;  
}
```

The guarantee depends on both threads coordinating through the same monitor.

It is not enough for one thread to synchronize while another thread accesses the same state without synchronization.

Ordering

21. What Is an Ordering Problem?

Ordering refers to the sequence in which memory operations become observable.

A programmer may write:

```
int data = 0;  
boolean ready = false;
```

Thread-1:

```
data = 10;  
ready = true;
```

Thread-2:

```
if (ready) {  
    System.out.println(data);  
}
```

```
}
```

The intended logic is:

```
Write data
→ mark data as ready
→ another thread observes ready
→ another thread reads data
```

Without synchronization, that ordering is not guaranteed between threads.

22. Why Reordering Happens

Compilers, the JVM, and processors may reorder instructions to improve performance.

They may perform optimizations such as:

- Instruction scheduling
- Register allocation
- Pipelining
- Store buffering
- Common-subexpression elimination

Reordering is permitted as long as the behavior remains valid according to single-thread semantics and the Java Memory Model.

In a single-threaded program, the result may appear unchanged.

In a multithreaded program without synchronization, another thread may observe operations in an unexpected order.

23. Classic Ordering Example

Initial state:

```
int data = 0;
```

```
boolean ready = false;
```

Thread-1:

```
data = 10;  
ready = true;
```

Thread-2:

```
if (ready) {  
    System.out.println(data);  
}
```

A programmer expects:

```
10
```

Without a happens-before relationship, Thread-2 is not guaranteed to observe the write to `data` merely because it observes `ready` as `true`.

It could observe:

```
ready = true  
data = 0
```

This may happen because of visibility effects, reordering, or both.

24. Preventing Ordering Problems with `volatile`

Declare the publication flag as volatile:

```
int data = 0;  
volatile boolean ready = false;
```

Thread-1:

```
data = 10;
ready = true;
```

Thread-2:

```
if (ready) {
    System.out.println(data);
}
```

The volatile write and read establish the required ordering:

```
Write data = 10
→ volatile write ready = true
→ volatile read ready == true
→ read data
```

If Thread-2 observes the volatile write to `ready`, it must also observe the preceding write to `data`.

This is a common safe-publication pattern.

25. Preventing Ordering Problems with `synchronized`

The same monitor can be used:

```
synchronized (lock) {
    data = 10;
    ready = true;
}
```

The reading thread must also acquire the same monitor:

```
synchronized (lock) {
    if (ready) {
        System.out.println(data);
    }
}
```

```
}  
}
```

The guarantee comes from:

```
Monitor release  
→ subsequent acquisition of the same monitor
```

It is more accurate to say that synchronization creates ordering and visibility guarantees between threads.

The JVM may still reorder internal instructions when doing so does not violate those guarantees.

Thread Interference

26. What Is Thread Interference?

Thread interference occurs when operations from multiple threads interleave in a way that causes one thread to disturb the work of another.

Example:

```
Thread-1 reads count  
Thread-2 reads count  
Thread-1 writes new value  
Thread-2 overwrites that value
```

Thread interference is usually the visible effect of:

- Shared mutable state
 - Non-atomic operations
 - Missing synchronization
 - Unsafe interleaving
-

27. Race Condition vs Thread Interference

The terms are related but emphasize different aspects.

Race Condition

A race condition describes the broader correctness problem:

The result depends on thread timing or ordering.

Thread Interference

Thread interference describes the harmful overlapping of operations:

One thread's operation disrupts another thread's operation.

A lost update is both:

- A result of thread interference
- An example of a race condition

Data Inconsistency

28. What Is Data Inconsistency?

Data inconsistency means:

The state stored by the program no longer matches its expected rules or real-world meaning.

Race conditions, visibility problems, ordering problems, and interference can all produce inconsistent data.

29. Common Forms of Data Inconsistency

Lost Update

```
count++;
```

Two threads perform an update, but one update is overwritten.

Stale Read

One thread observes an older value because the latest update is not visible.

```
while (!flag) {  
    // may continue using stale value  
}
```

Inconsistent Read

A thread observes related fields from different logical moments.

Example:

```
accountBalance reflects the new state  
transactionStatus still reflects the old state
```

Partial Update

Only part of a multi-step operation is visible or completed.

Example:

```
Money deducted from Account A  
but not yet credited to Account B
```

Check-Then-Act Failure

A thread acts on a condition that was true when checked but became false before the action.

```
if (balance >= amount) {  
    balance -= amount;  
}
```

Unsafe Publication

A thread receives a reference to an object before the object's state has been safely made visible.

The receiving thread may observe default or stale field values.

Preventing Multithreading Problems

30. Using `synchronized`

```
synchronized (lock) {  
    count++;  
}
```

Use synchronization when:

- Several operations must execute as one unit
- Multiple fields form one consistency rule
- Mutual exclusion and visibility are both required
- Threads must later coordinate using `wait()` and `notifyAll()`

31. Using Atomic Classes

```
AtomicInteger count = new AtomicInteger();  
  
count.incrementAndGet();
```

Common atomic classes include:

`AtomicInteger`
`AtomicLong`
`AtomicBoolean`
`AtomicReference`

They are useful for simple atomic state transitions without protecting a larger compound operation.

32. Using Explicit Locks

```
Lock lock = new ReentrantLock();

lock.lock();

try {
    count++;
} finally {
    lock.unlock();
}
```

Explicit locks provide features such as:

- Interruptible lock acquisition
- Timed lock attempts
- Fairness policies
- Multiple condition objects

They require manual release, usually inside `finally`.

33. Using Thread-Safe Collections

Examples:

ConcurrentHashMap
CopyOnWriteArrayList
BlockingQueue
ConcurrentLinkedQueue

These collections are designed for concurrent access.

The correct choice depends on whether the workload is read-heavy, write-heavy, blocking, or non-blocking.

34. Using Immutability

Immutable objects do not change after creation.

```
final class User {  
    private final String name;  
    private final int age;  
  
    User(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
}
```

Because the state cannot change, multiple threads can safely read a properly constructed and safely published immutable object.

Immutability removes many synchronization requirements by design.

35. Avoiding Shared Mutable State

The safest shared mutable state is often the state that is not shared.

Useful techniques include:

- Thread-local variables
- Method-local variables

- Message passing
- Immutable snapshots
- Task confinement
- Dividing work so each thread owns separate data

Good concurrent design often reduces sharing before adding locks.

Comparison of the Main Problems

Problem	Main Meaning	Typical Example
Race condition	Result depends on thread timing	Two threads increment the same counter
Atomicity problem	A multi-step operation is interrupted or interleaved	<code>count++</code>
Visibility problem	One thread does not observe another thread's write	Non-volatile stop flag
Ordering problem	Operations are observed in an unexpected order	Publication flag seen before data
Thread interference	Threads overlap and disturb each other's work	Lost update
Data inconsistency	Program state violates expected rules	Incorrect balance or stale data

Choosing the Correct Tool

Requirement	Suitable Mechanism
Simple visible status flag	<code>volatile</code>
Atomic counter update	<code>AtomicInteger</code>
Several statements must be atomic together	<code>synchronized</code> or <code>Lock</code>
Concurrent map access	<code>ConcurrentHashMap</code>
Producer-consumer communication	<code>BlockingQueue</code> or <code>wait()</code> / <code>notifyAll()</code>

Requirement	Suitable Mechanism
State that should never change	Immutable object
Avoid sharing entirely	Thread confinement or message passing

Important Rules

1. Shared mutable data is the primary source of concurrency problems.
2. `count++` is not atomic even though it is one Java statement.
3. A race condition depends on unpredictable thread timing.
4. Atomicity prevents unsafe interleaving of a logical operation.
5. Visibility determines whether one thread can observe another thread's writes.
6. Ordering determines how operations become observable between threads.
7. `volatile` provides visibility and ordering guarantees but does not make compound operations atomic.
8. `synchronized` provides mutual exclusion, visibility, and ordering through monitor boundaries.
9. All threads must follow the same synchronization policy.
10. Thread-safe collections protect their own operations, but a sequence of calls may still require atomic coordination.
11. Immutable objects and reduced sharing simplify concurrent design.
12. Correct concurrent code should be based on Java Memory Model guarantees, not assumptions about CPU caches or execution timing.

Final Summary

The major multithreading problems are connected:

```
Shared mutable state
+
multiple threads
```

Coder Army

```
+  
missing coordination  
↓  
unsafe interleaving  
↓  
race conditions and thread interference  
↓  
lost updates, stale reads, ordering failures  
↓  
data inconsistency
```

The main mechanisms used to solve these problems are:

```
synchronized  
volatile  
Atomic classes  
Explicit locks  
Thread-safe collections  
Immutability  
Reduced sharing
```

The most important distinction is:

```
Atomicity → can another thread interfere halfway?  
Visibility → can another thread see the latest value?  
Ordering → in what order do writes become observable?
```

Understanding these three guarantees is the foundation of thread-safe Java programming.